



# Red Hat Lightspeed cost management API

Pau Garcia Quiles, Senior Principal Product Manager, Red Hat Lightspeed

# Table of Contents

|                                                                |   |
|----------------------------------------------------------------|---|
| Introduction .....                                             | 1 |
| Authentication .....                                           | 1 |
| Using service account tokens (OAuth2 client credentials) ..... | 1 |
| OpenShift Costs .....                                          | 1 |
| Cluster costs .....                                            | 1 |
| Project costs .....                                            | 2 |
| Project costs grouped by cluster .....                         | 2 |
| Project costs grouped by tag .....                             | 2 |
| Node costs .....                                               | 2 |
| Tag costs .....                                                | 2 |
| OpenShift Virtualization VMs .....                             | 2 |
| CPU .....                                                      | 2 |
| Memory .....                                                   | 2 |
| Storage volumes .....                                          | 3 |
| GPU costs .....                                                | 3 |
| Breakdown page: costs for a cluster .....                      | 3 |
| Breakdown page: costs for a project .....                      | 3 |
| Cloud Filtered by OpenShift .....                              | 3 |
| AWS Costs .....                                                | 4 |
| Per AWS account .....                                          | 4 |
| Per AWS region .....                                           | 4 |
| Per AWS service .....                                          | 4 |
| Per AWS EC2 VM .....                                           | 4 |
| Per AWS tag .....                                              | 4 |
| Per AWS cost category .....                                    | 4 |
| Per AWS Organizational Unit .....                              | 4 |
| Per AWS instance type .....                                    | 5 |
| Per AWS storage .....                                          | 5 |
| AWS cost types .....                                           | 5 |
| Azure Costs .....                                              | 5 |
| Per Azure account (subscription) .....                         | 5 |
| Per Azure region .....                                         | 5 |
| Per Azure service .....                                        | 5 |
| Per Azure tag .....                                            | 5 |
| Per Azure instance type .....                                  | 5 |
| Per Azure storage .....                                        | 6 |
| Google Cloud Costs .....                                       | 6 |
| Per Google Cloud account .....                                 | 6 |
| Per Google Cloud region .....                                  | 6 |
| Per Google Cloud service .....                                 | 6 |
| Per Google Cloud Project .....                                 | 6 |
| Per Google Cloud tag .....                                     | 6 |

|                                                           |    |
|-----------------------------------------------------------|----|
| Per Google Cloud instance type .....                      | 6  |
| Per Google Cloud storage .....                            | 6  |
| Tags .....                                                | 6  |
| List tag keys and values .....                            | 7  |
| Optimizations (Resource Optimization for OpenShift) ..... | 7  |
| Notification codes (reference) .....                      | 7  |
| Fleet summary .....                                       | 7  |
| Container recommendations .....                           | 9  |
| Namespace recommendations .....                           | 12 |
| GPU recommendations hub (summary) .....                   | 13 |
| Node CPU/memory utilization recommendations .....         | 14 |
| MachineSet aggregation recommendations .....              | 15 |
| PVC (Persistent Volume Claim) recommendations .....       | 16 |
| ResourceQuota recommendations .....                       | 16 |
| ClusterResourceQuota recommendations .....                | 19 |
| Snapshot staleness recommendations .....                  | 22 |
| OpenShift Virtualization (VM) recommendations .....       | 22 |
| Recommendation term settings .....                        | 30 |
| Settings capabilities .....                               | 30 |
| Platform settings lock .....                              | 31 |
| Recommendation threshold settings .....                   | 31 |
| Business hours settings .....                             | 34 |
| Idle and zombie detection .....                           | 35 |
| Recommendation history and quality .....                  | 39 |
| Forecasts .....                                           | 39 |
| OpenShift cost forecast .....                             | 39 |
| AWS cost forecast .....                                   | 39 |
| Azure cost forecast .....                                 | 39 |
| Google Cloud cost forecast .....                          | 39 |
| OCP-on-cloud cost forecasts .....                         | 39 |
| Resource Types .....                                      | 40 |
| OpenShift .....                                           | 40 |
| AWS .....                                                 | 40 |
| Azure .....                                               | 40 |
| Google Cloud .....                                        | 40 |
| Cost models .....                                         | 40 |
| Cost Distribution Fields .....                            | 41 |
| Settings .....                                            | 41 |
| Platform projects (cost groups) .....                     | 41 |
| Enabled tags .....                                        | 41 |
| Display currency .....                                    | 41 |
| Cost models .....                                         | 42 |
| Integrations .....                                        | 42 |
| Filtering and Pagination .....                            | 42 |

|                               |    |
|-------------------------------|----|
| Time scope filters .....      | 42 |
| Exclude filter .....          | 43 |
| Pagination.....               | 43 |
| Get CSV or JSON Reports ..... | 43 |
| Basic Field Definitions.....  | 44 |

# Introduction

The Cost Management API allows users to obtain all the data available in the UI, and even more since with the API it's possible to combine filters or to make several API calls and then combine the results.

The latest version of this cheat sheet, and its accompanying Bruno collection, are always available from

<https://github.com/project-koku/costmgmt-api-cheatsheet>.

Below are common scenarios with example API calls.

Base URL: <https://console.redhat.com>

## Authentication

### Using service account tokens (OAuth2 client credentials)

Grab your token using the following curl command:

```
curl -v \
  -d "client_id=CLIENT_ID" \
  -d "client_secret=CLIENT_SECRET" \
  -d "grant_type=client_credentials" \
  -d "scope=api.console" \
  "https://sso.redhat.com/auth/realms/redhat-external/protocol/openid-connect/token"
```

Use the token in an API request via curl:

```
curl --location \
  'https://console.redhat.com/api/cost-management/v1/reports/aws/costs/' \
  --header 'Authorization: Bearer ACCESS_TOKEN'
```

## OpenShift Costs

### Cluster costs

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[cluster]=*
```

Cluster costs only for the current month:

```
GET /api/cost-management/v1/reports/openshift/costs/?filter[time_scope_units]=month&group_by[cluster]=*
```

## Project costs

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[project]=*
```

## Project costs grouped by cluster

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[cluster]=*&group_by[project]=*
```

## Project costs grouped by tag

Costs by project, grouped by a tag key (e.g. `application`):

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[project]=*&group_by[tag:application]=*
```

## Node costs

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[node]=*
```

## Tag costs

Costs grouped by a tag key (e.g. `env`):

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[tag:env]=*
```

## OpenShift Virtualization VMs

```
GET /api/cost-management/v1/reports/openshift/resources/virtual-machines/
```

Optionally filter by cluster:

```
GET /api/cost-management/v1/reports/openshift/resources/virtual-machines/?filter[exact:cluster]=CLUSTER_UUID
```

## CPU

```
GET /api/cost-management/v1/reports/openshift/compute/
```

## Memory

```
GET /api/cost-management/v1/reports/openshift/memory/
```

## Storage volumes

```
GET /api/cost-management/v1/reports/openshift/volumes/?group_by[storageclass]=*
```

Storage volumes for a specific project (e.g. `cost-management`):

```
GET /api/cost-management/v1/reports/openshift/volumes/?filter[exact:project]=cost-management&group_by[storageclass]=*
```

## GPU costs

```
GET /api/cost-management/v1/reports/openshift/gpu/
```

GPU costs by MIG profile:

```
GET /api/cost-management/v1/reports/openshift/gpu/mig_profiles/
```

## Breakdown page: costs for a cluster

Cost breakdown for the "demolab" cluster:

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[cluster]=demolab
```

## Breakdown page: costs for a project

Cost breakdown for the "cost-management" project:

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[project]=cost-management
```

## Cloud Filtered by OpenShift

The following endpoints return cloud data specifically relating to OpenShift clusters. They allow for both cluster and cloud group\_by/filtering options (cluster, node, project, account, service, region).

```
GET /api/cost-management/v1/reports/openshift/infrastructures/all/costs/  
GET /api/cost-management/v1/reports/openshift/infrastructures/aws/costs/  
GET /api/cost-management/v1/reports/openshift/infrastructures/azure/costs/  
GET /api/cost-management/v1/reports/openshift/infrastructures/gcp/costs/
```

# AWS Costs

## Per AWS account

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[account]=*
```

## Per AWS region

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[region]=*
```

## Per AWS service

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[service]=*
```

## Per AWS EC2 VM

```
GET /api/cost-management/v1/reports/aws/resources/ec2-compute/
```

Filtering by operating system (e.g. Fedora):

```
GET /api/cost-management/v1/reports/aws/resources/ec2-compute/?filter[operating_system]=Fedora
```

## Per AWS tag

Costs grouped by an AWS tag key (e.g. `AppTeam`):

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[tag:AppTeam]=*
```

## Per AWS cost category

Costs grouped by an AWS cost category (e.g. `CostCenter`):

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[aws_category:CostCenter]=*
```

## Per AWS Organizational Unit

Group by root organizational unit "r-f22a":

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[org_unit_id]=r-f22a
```

## Per AWS instance type

```
GET /api/cost-management/v1/reports/aws/instance-types/
```

## Per AWS storage

```
GET /api/cost-management/v1/reports/aws/storage/
```

## AWS cost types

Getting unblended, blended and amortized costs:

```
GET /api/cost-management/v1/reports/aws/costs/?cost_type=unblended_cost
GET /api/cost-management/v1/reports/aws/costs/?cost_type=blended_cost
GET /api/cost-management/v1/reports/aws/costs/?cost_type=savingsplan_effective_cost
```

## Azure Costs

### Per Azure account (subscription)

```
GET /api/cost-management/v1/reports/azure/costs/?group_by[subscription_guid]=*
```

### Per Azure region

```
GET /api/cost-management/v1/reports/azure/costs/?group_by[resource_location]=*
```

### Per Azure service

```
GET /api/cost-management/v1/reports/azure/costs/?group_by[service_name]=*
```

### Per Azure tag

Costs grouped by an Azure tag key (e.g. `ms-resource-usage`):

```
GET /api/cost-management/v1/reports/azure/costs/?group_by[tag:ms-resource-usage]=*
```

### Per Azure instance type

```
GET /api/cost-management/v1/reports/azure/instance-types/
```

## Per Azure storage

```
GET /api/cost-management/v1/reports/azure/storage/
```

## Google Cloud Costs

### Per Google Cloud account

```
GET /api/cost-management/v1/reports/gcp/costs/?group_by[account]=*
```

### Per Google Cloud region

```
GET /api/cost-management/v1/reports/gcp/costs/?group_by[region]=*
```

### Per Google Cloud service

```
GET /api/cost-management/v1/reports/gcp/costs/?group_by[service]=*
```

### Per Google Cloud Project

```
GET /api/cost-management/v1/reports/gcp/costs/?group_by[gcp_project]=*
```

### Per Google Cloud tag

Costs grouped by a Google Cloud label (e.g. `env`):

```
GET /api/cost-management/v1/reports/gcp/costs/?group_by[tag:env]=*
```

### Per Google Cloud instance type

```
GET /api/cost-management/v1/reports/gcp/instance-types/
```

### Per Google Cloud storage

```
GET /api/cost-management/v1/reports/gcp/storage/
```

## Tags

## List tag keys and values

Cloud source tags:

```
GET /api/cost-management/v1/tags/aws/  
GET /api/cost-management/v1/tags/azure/  
GET /api/cost-management/v1/tags/gcp/
```

OpenShift tags (labels):

```
GET /api/cost-management/v1/tags/openshift/
```

You can also do OCP-on-cloud variations by adding `infrastructures/{cloud}` (i.e. `gcp`, `azure`, `aws`).

## Additional filtering options

Show disabled tags:

```
GET /api/cost-management/v1/tags/openshift/?filter[enabled]=False
```

Show tags for a particular time frame:

```
GET /api/cost-management/v1/tags/openshift/?start_date=2025-11-15&end_date=2025-12-17
```

See tag values directly for a specific key by appending the key ("my-tag-key") to the URL:

```
GET /api/cost-management/v1/tags/openshift/my-tag-key/
```

# Optimizations (Resource Optimization for OpenShift)

## Notification codes (reference)

Machine-readable catalog of all notification codes (no auth required). Optional `filter[plugin]` for `container`, `namespace`, `node`, `gpu`, `pvc`, `snapshot`, `vm`, `quota`, or `cluster-quota`.

```
GET /api/cost-management/v1/recommendations/openshift/notification-codes  
GET /api/cost-management/v1/recommendations/openshift/notification-codes?filter[plugin]=container  
GET /api/cost-management/v1/recommendations/openshift/notification-codes?filter[plugin]=namespace
```

## Fleet summary

Cross-cluster aggregated savings, adoption rates, and top optimization opportunities:

```
GET /api/cost-management/v1/recommendations/openshift/fleet-summary
```

## Fleet savings summary

Aggregated monthly savings by plugin (container, node, PVC, snapshot, VM). Container, node, and VM totals honor `?engine=cost` (default) or `performance`. GPU dollar estimates are computed at API read time and are excluded from this rollup (`by_plugin.gpu` is always `0`; see `gpu_savings_note`).

Use `?term=short`, `?term=medium` (default), or `?term=long` to align node/container/PVC rollups with the same term window as list APIs. VM savings use mapped term keys (`short_term`, `medium_term`, `long_term`).

Requires persisted recommendations for the selected term. VM rows need `ROS_SAVINGS_ESTIMATES_ENABLED=true` and `KOKU_MASU_URL` at ingestion time for non-zero `by_plugin.vm`.

```
GET /api/cost-management/v1/recommendations/openshift/savings-summary
GET /api/cost-management/v1/recommendations/openshift/savings-summary?engine=cost
GET /api/cost-management/v1/recommendations/openshift/savings-summary?engine=cost&term=short
GET /api/cost-management/v1/recommendations/openshift/savings-summary?filter[cluster]=<cluster-uuid>&group_by[tag:environment]=*
GET /api/cost-management/v1/recommendations/openshift/savings-summary?group_by[tag:environment]=*
```

### NOTE

`filter[cluster]` is only applied when a `group_by` parameter is present (e.g., `group_by[tag:*)` or `group_by[idle_state]`). Without `group_by`, the filter has no effect and all RBAC-visible clusters are included.

## Savings recalculation (internal)

After a Koku cost model update, masu notifies ROS to refresh persisted dollar savings without re-ingestion.

Supported types: `container`, `node`, `pvc`, `quota`, `cluster-quota`. Service-account bearer token required (same auth as tag sync). Requires `ROS_SAVINGS_ESTIMATES_ENABLED` and `ROS_SAVINGS_RECALCULATION_ENABLED`.

```
POST /api/cost-management/v1/internal/recalculate-savings
```

Example body:

```
{
  "org_id": "1234567",
  "cluster_uuid": "02059694-68ab-4d58-8809-de1e91f1d0e5",
  "recommendation_types": ["container", "node", "pvc", "quota", "cluster-quota"]
}
```

Returns `202 Accepted` with `status: "accepted"`; work runs asynchronously.

Example response excerpt:

```
{
  "currency": "USD",
```

```

    "estimated_monthly_savings": { "value": "12500.750000", "units": "USD" },
    "by_plugin": {
      "container": 5000.0,
      "node": 3000.0,
      "pvc": 1500.0,
      "snapshot": 500.0,
      "vm": 1200.0,
      "gpu": 0.0
    },
    "gpu_savings_note": "GPU savings are computed at API read time..."
  }
}

```

## Container recommendations

All container-level CPU/memory right-sizing recommendations:

```
GET /api/cost-management/v1/recommendations/openshift/
```

Recommendations for a specific container (use the recommendation UUID):

```
GET /api/cost-management/v1/recommendations/openshift/{recommendation_uuid}
```

The detail response includes per-term (short/medium/long) recommendations for two models:

- **Cost model:** optimizes for resource savings (p60 CPU / p95 memory)
- **Performance model:** optimizes for headroom (p98 CPU / p100 memory)

Each recommendation includes CPU/memory requests and limits, notifications, and usage distribution box plots (min, Q1, median, Q3, max).

Detail responses nest both engines under each term:

```

recommendations.recommendation_terms.medium_term.recommendation_engines.cost
recommendations.recommendation_terms.medium_term.recommendation_engines.performance

```

## How engines differ (cost vs performance)

- **Cost** – lower usage percentiles (e.g. P60 CPU, P95 memory) to right-size for savings
- **Performance** – higher percentiles (e.g. P98 CPU, max memory) to preserve headroom

On list endpoints, `filter[engine]=cost` or `filter[engine]=performance` returns only the selected engine under each term and omits the other from `recommendation_engines`. When the filter is omitted, both engines are returned.

To generate cluster data where cost and performance sizing intentionally diverge (for validation or demos), use the NISE fixture at `nise/examples/ocp_dual_engine/` (`spike-cpu-api`, `steady-mem-worker` workloads; see the README in that directory).

## Testing divergence

After ingesting `nise/examples/ocp_dual_engine/` data, container cost vs performance sizing will differ under `recommendation_terms.*.recommendation_engines` (higher CPU/memory on performance for spike-prone workloads). Use that fixture in your test cluster to verify dual-engine divergence in E2E, IQE, or Bruno manual runs; generic fixtures may return identical sizing while still exposing both engine blocks.

## Filtering and sorting

Filter by cluster, project, workload, workload type, or container:

```
GET /api/cost-management/v1/recommendations/openshift/?filter[cluster]=CLUSTER_UUID
GET /api/cost-management/v1/recommendations/openshift/?filter[project]=my-namespace
GET /api/cost-management/v1/recommendations/openshift/?filter[namespace]=my-namespace
GET /api/cost-management/v1/recommendations/openshift/?filter[workload]=my-deployment
GET /api/cost-management/v1/recommendations/openshift/?filter[workload_type]=deployment
GET /api/cost-management/v1/recommendations/openshift/?filter[container]=my-container
```

`filter[namespace]` is an alias for `filter[project]` (both filter by OpenShift namespace). Legacy flat forms (`cluster_uuid`, `project`, `namespace`, `container`) are also accepted.

`filter[workload_type]` accepts lowercase Kubernetes controller kinds (exact match; comma-separated values are ORed): `daemonset`, `deployment`, `deploymentconfig`, `replicaset`, `replicationcontroller`, `statefulset`. Legacy flat form `?workload_type=deployment` is also accepted.

Filter by recommendation engine (limits nested `recommendation_engines` to cost or performance; omits the non-selected engine from each item):

```
GET /api/cost-management/v1/recommendations/openshift/?filter[engine]=cost
GET /api/cost-management/v1/recommendations/openshift/?filter[engine]=performance
```

Legacy flat `?engine=cost` is also accepted on container, namespace, node, VM, quality, and history endpoints.

Filter by GPU:

```
GET /api/cost-management/v1/recommendations/openshift/?filter[has_gpu]=true
GET /api/cost-management/v1/recommendations/openshift/?filter[gpu_model]=A100
GET /api/cost-management/v1/recommendations/openshift/?filter[gpu_classification]=underutilized
```

Filter by idle state (`active`, `idle`, or `zombie`; comma-separated values are ORed):

**NOTE** The API uses `idle_state=zombie` which the UI displays as "Abandoned".

```
GET /api/cost-management/v1/recommendations/openshift/?filter[idle_state]=zombie,idle
```

Filter by OpenShift labels (requires `ROS_TAGS_ENABLED=true`; tags synced from Koku):

```
GET /api/cost-management/v1/recommendations/openshift/?filter[tag:environment]=production
GET /api/cost-management/v1/recommendations/openshift/?filter[tag:environment]=production,staging
GET /api/cost-management/v1/recommendations/openshift/?filter[tag:team]=platform
```

Tag filter syntax follows Koku report conventions:

- `filter[tag:<key>]=<value>` – exact value match
- `filter[tag:<key>]=a,b` – OR within the same key (comma-separated)
- Multiple `filter[tag:*]` keys – AND across keys
- `filter[tag:<key>]=*` – key present with any value
- Legacy flat form `?tag=<key>:<value>` is also accepted

When a tag key is unknown or has no matching containers, `meta.count` is `0` and `meta.warnings` may describe the empty result.

## Pagination

Offset pagination (default):

```
GET /api/cost-management/v1/recommendations/openshift/?limit=20&offset=0
```

Keyset pagination (stable iteration; `offset` is ignored when `after` is set):

```
GET /api/cost-management/v1/recommendations/openshift/?limit=20
# Response meta includes has_next and next_cursor when more pages exist
GET /api/cost-management/v1/recommendations/openshift/?limit=20&after=<meta.next_cursor>
```

## Sorting

Allowed `order_by` values: `cluster`, `project`, `workload_type`, `workload`, `container`, `last_reported`, `cpu_request_current`, `memory_request_current`, and per-term variation fields: `cpu_variation_short_cost`, `cpu_variation_short_performance`, `cpu_variation_medium_cost`, `cpu_variation_medium_performance`, `cpu_variation_long_cost`, `cpu_variation_long_performance`, `memory_variation_short_cost`, `memory_variation_short_performance`, `memory_variation_medium_cost`, `memory_variation_medium_performance`, `memory_variation_long_cost`, `memory_variation_long_performance`.

```
GET /api/cost-management/v1/recommendations/openshift/?order_by=last_reported&order_how=desc
GET /api/cost-management/v1/recommendations/openshift/?order_by=cpu_variation_medium_cost&order_how=desc
```

## CSV export

Export container recommendations as CSV (one row per container × term × engine):

```
GET /api/cost-management/v1/recommendations/openshift/?format=csv
```

```
GET /api/cost-management/v1/recommendations/openshift/?format=csv&limit=100&order_by=last_reported&order_how=desc
```

Prefer `Accept: text/csv` for explicit content negotiation. Response `Content-Type` is `text/csv`. The export is capped at 1000 list records (up to 6000 CSV rows when all three terms and both engines are present). Columns include cluster, project, workload, container, savings, idle state, and per-term CPU/memory requests and limits. Use the same filters and `order_by` parameters as the JSON list endpoint.

## History and quality

Node recommendations do not expose a history or quality API. Per-node history is N/A; `GET .../quality` applies to container recommendations only (stability, adoption, OOM after rec).

Recommendation snapshot history (fleet-wide, native engine – containers):

```
GET /api/cost-management/v1/recommendations/openshift/history?limit=10
GET /api/cost-management/v1/recommendations/openshift/history?container=my-container&filter[engine]=cost&term=medium
```

History accepts `filter[engine]=cost|performance` (preferred) or legacy flat `?engine=cost`.

Recommendation quality metrics (stability, adoption, OOM after rec):

```
GET /api/cost-management/v1/recommendations/openshift/quality?limit=10
GET /api/cost-management/v1/recommendations/openshift/quality?filter[engine]=cost&limit=10
GET /api/cost-management/v1/recommendations/openshift/quality?order_by=stability&order_how=desc
```

Quality defaults to the cost engine when `filter[engine]` is omitted.

## Replica counts and total savings

Container detail responses include replica information collected from `kube-state-metrics` (deployments, statefulsets, daemonsets):

- `replica_info.desired` – spec-level replica count
- `replica_info.available` – currently available/ready replicas
- `replica_info.source` – `"kube_state_metrics"` (authoritative) or `"derived"` (pod count fallback)

Total impact savings multiply per-container savings by replica count.

## Namespace recommendations

Namespace-level CPU/memory recommendations (aggregated across all containers in a namespace):

```
GET /api/cost-management/v1/recommendations/openshift/namespaces
GET /api/cost-management/v1/recommendations/openshift/namespaces?limit=100&offset=0
```

```
GET /api/cost-management/v1/recommendations/openshift/namespaces?filter[stale]=false
GET /api/cost-management/v1/recommendations/openshift/namespaces?filter[stale]=true
```

Canonical list path. Legacy alias (identical behavior): `GET /api/cost-management/v1/openshift/namespace/recommendations`.

- `filter[stale]` – `false` (default, exclude stale clusters), `true` (include stale and fresh), or `only` (only stale). Stale when the cluster has not reported within `ROS_STALENESS_THRESHOLD_HOURS` (default 48).
- `filter[engine]` – `cost` or `performance`; omits the other engine from `recommendation_engines` on each item (legacy `?engine=` also accepted)

```
GET /api/cost-management/v1/recommendations/openshift/namespaces?filter[engine]=cost
GET /api/cost-management/v1/recommendations/openshift/namespaces?filter[engine]=performance
```

Detail for a specific namespace recommendation:

```
GET /api/cost-management/v1/recommendations/openshift/namespaces/{recommendation_uuid}
```

History for a namespace recommendation:

```
GET /api/cost-management/v1/recommendations/openshift/namespaces/{recommendation_uuid}/history?filter[term]=short_term&filter[engine]=cost&limit=10
```

## GPU recommendations hub (summary)

Lightweight counts plus absolute links to the time-slicing and MIG listing endpoints:

```
GET /api/cost-management/v1/recommendations/openshift/gpu
```

## NVIDIA MIG (Multi-Instance GPU) recommendations

MIG bin-packing recommendations appear in the container detail response and in a dedicated list endpoint.

Detail response – when a container uses a MIG-capable GPU (A100, H100, H200, B200), the response includes a `gpu` object with:

- `current_gpu_model` / `current_gpu_profile` – what the container currently uses
- `recommended_gpu_profile` – smallest MIG profile whose resources cover p95 usage
- `gpu_classification` – e.g. `compute_bound`, `memory_bound`, `underutilized`
- `estimated_monthly_gpu_savings_usd` – dollar savings from MIG right-sizing

```
GET /api/cost-management/v1/recommendations/openshift/{recommendation_uuid}
```

List containers that have a non-full-GPU MIG recommendation:

```
GET /api/cost-management/v1/recommendations/openshift/gpu/mig
GET /api/cost-management/v1/recommendations/openshift/gpu/mig?limit=20&offset=0
GET /api/cost-management/v1/recommendations/openshift/gpu/mig?order_by=estimated_monthly_gpu_savings_usd&order_how=desc
```

Supports pagination (`limit`, `offset`, capped by the API max), sorting (`order_by`, `order_how`), and filters consistent with the OpenAPI spec for this route.

## NVIDIA GPU time-slicing recommendations

Node-level GPU time-slicing consolidation guidance. Evaluates how many containers could share each physical GPU via `nvidia.com/gpu.replicas`:

```
GET /api/cost-management/v1/recommendations/openshift/gpu/timeslicing
GET /api/cost-management/v1/recommendations/openshift/gpu/timeslicing?limit=25&offset=0
GET /api/cost-management/v1/recommendations/openshift/gpu/timeslicing?node_name=my-gpu-node&gpu_model=T4
```

Returns per-node recommendations with:

- `gpu_model` – NVIDIA GPU model on the node
- `recommended_replicas` – suggested time-slicing replica count
- `savings_per_gpu_usd` / `total_node_savings_usd` – estimated cost savings

## Node CPU/memory utilization recommendations

Node-level right-sizing based on CPU and memory utilization patterns – identifies underutilized, overcommitted, stranded-resource, idle, and zombie nodes. One JSON object per node with nested `recommendation_terms` and dual engines (`cost`, `performance`).

Business hours are not applicable to node recommendations (they apply to container and namespace recommendations only).

```
GET /api/cost-management/v1/recommendations/openshift/nodes
GET /api/cost-management/v1/recommendations/openshift/nodes?limit=10&offset=0
GET /api/cost-management/v1/recommendations/openshift/nodes?filter[term]=medium&filter[engine]=cost
GET /api/cost-management/v1/recommendations/openshift/nodes?filter[cluster]=CLUSTER_UUID&filter[node]=worker-0&limit=1
GET /api/cost-management/v1/recommendations/openshift/nodes?format=csv
```

List filters (bracket syntax preferred): `filter[cluster]`, `filter[node]`, `filter[term]` (`short` | `medium` | `long`), `filter[engine]` (`cost` | `performance`), `filter[is_underutilized]`, `filter[is_overcommitted]`, `filter[idle_state]`

(comma-separated `active`, `idle`, `zombie`), `filter[stranded_resource]` (`cpu`, `memory`, `none`), `filter[instance_type]`, `filter[machineset_name]`, `filter[tag:<key>=<value>` (requires `ROS_TAGS_ENABLED=true`; nodes match when any workload namespace on the cluster has the label). Sort: `order_by` (`estimated_monthly_savings` default, or `node`), `order_how` (`asc` | `desc`). Pagination: `limit`, `offset`. Legacy flat params `?term=`, `?engine=`, `?cluster_uuid=` still work.

RBAC: node list and detail require `cost-management.openshift.node` permission (node name scope) in addition to cluster access via `cost-management.openshift.cluster`. Empty results when the user has no intersecting cluster/node permissions.

Single-node detail: `GET .../nodes/{node}?filter[cluster]=CLUSTER_UUID` (registered). List-filter equivalent: `filter[node]` + `filter[cluster]` with `limit=1` (same response shape).

```
GET /api/cost-management/v1/recommendations/openshift/nodes?filter[is_underutilized]=true
GET /api/cost-management/v1/recommendations/openshift/nodes?filter[stranded_resource]=memory
GET /api/cost-management/v1/recommendations/openshift/nodes?filter[machineset_name]=worker
GET /api/cost-management/v1/recommendations/openshift/nodes?filter[tag:environment]=production
```

Response fields (per node): `classification` (including `idle_state`, `stranded_resource`), `metrics` (CPU/memory P50/P95), `pod_count`, `pod_capacity`, `pod_scheduling_headroom`, `instance_type`, `machineset_name`, `suggested_instance_type`, `instance_type_reason`, `cpu_overcommit_ratio`, `trend_slope`, nested `recommendation_terms.<term>.recommendation_engines.{cost,performance}` with sizing, `node_count_reduction`, savings, and notifications (code **13** includes `suggested_direction` for stranded CPU/memory).

CSV export (`?format=csv` or `Accept: text/csv`): one row per node × term × engine; columns include `pod_capacity`, `pod_scheduling_headroom`, classification, and savings.

Fleet consolidation groups underutilized nodes by `instance_type` when the operator provides it; otherwise by similar allocatable CPU/memory capacity.

## MachineSet aggregation recommendations

Groups per-node recommendations by OpenShift MachineSet name (Tier 2 list API; aggregation only – no separate engine table):

```
GET /api/cost-management/v1/recommendations/openshift/machinesets
GET /api/cost-management/v1/recommendations/openshift/machinesets?filter[term]=medium&limit=10&offset=0
GET /api/cost-management/v1/recommendations/openshift/machinesets?filter[cluster]=CLUSTER_UUID
GET /api/cost-management/v1/recommendations/openshift/machinesets?filter[machineset_name]=worker*
```

Filters: `filter[cluster]` / `cluster_uuid`, `filter[machineset_name]` (exact or `*` wildcard), `filter[term]` / `term` (`short` | `medium` | `long`, default `medium`). Sorted by `total_monthly_savings_usd` descending.

Response fields (per MachineSet): `machineset_name`, `cluster_uuid`, `cluster_alias`, `instance_type`, `current_node_count`, `recommended_node_count`, `excess_nodes`, `total_monthly_savings_usd`, `avg_cpu_utilization`, `avg_memory_utilization`, `nodes`.

MachineAutoscaler integration is planned as a future enhancement (Tier 3).

## Deprecated path (backward compatibility)

Historically, `/recommendations/openshift/nodes` served GPU time-slicing listings; that behavior now lives under `/recommendations/openshift/gpu/timeslicing`. The old CPU/memory listing path was `/recommendations/openshift/nodes/utilization`.

```
GET /api/cost-management/v1/recommendations/openshift/nodes/utilization
```

**Deprecated** alias for `GET /api/cost-management/v1/recommendations/openshift/nodes`. Prefer the canonical `/nodes` URL. Responses include `Deprecation: true` and a `Link` header to `/nodes`. IQE and new integrations should call `/nodes` only.

## PVC (Persistent Volume Claim) recommendations

Right-sizing recommendations for persistent volumes – oversized, near-full, orphaned, and growth trend projection:

```
GET /api/cost-management/v1/recommendations/openshift/pvcs
```

## ResourceQuota recommendations

Namespace-level Kubernetes `ResourceQuota` right-sizing (distinct from namespace CPU/memory usage recommendations). Compares configured quota hard limits and used consumption against aggregated container recommendation totals.

Quota savings (`estimated_savings`) are per-recommendation only. They are NOT included in the fleet `GET /savings-summary` rollup (which covers container, node, PVC, snapshot, and VM plugins).

```
GET /api/cost-management/v1/recommendations/openshift/quota/
GET /api/cost-management/v1/recommendations/openshift/quota/?filter[cluster]=550e8400-e29b-41d4-a716-446655440000
GET /api/cost-management/v1/recommendations/openshift/quota/?filter[project]=production
GET /api/cost-management/v1/recommendations/openshift/quota/?filter[quota_name]=compute-quota
GET /api/cost-management/v1/recommendations/openshift/quota/?filter[recommendation_type]=tighten&filter[risk_level]=low
GET /api/cost-management/v1/recommendations/openshift/quota/?order_by=utilization&order_how=desc&limit=20&offset=0
GET /api/cost-management/v1/recommendations/openshift/quota/?group_by[cluster]=*&limit=20&offset=0
GET /api/cost-management/v1/recommendations/openshift/quota/?group_by[project]=*&limit=20&offset=0
GET /api/cost-management/v1/recommendations/openshift/quota/detail?cluster_uuid=550e8400-e29b-41d4-a716-446655440001&namespace=production&quota_name=compute-quota
```

## Available filters:

- `filter[cluster]` – cluster UUID (exact match)
- `filter[project]` – namespace name (exact match)
- `filter[quota_name]` – ResourceQuota object name (alias `filter[resource_quota_name]`)
- `filter[recommendation_type]` – `tighten`, `raise`, `optimal`, or `none` (no recommendation generated yet)
- `filter[risk_level]` – `high`, `medium`, `low`, or `none`
- `group_by[cluster]` or `group_by[project]` – aggregate rows (mutually exclusive)
- `order_by` – `quota_name`, `utilization`, `risk_level`, `namespace`, or `estimated_monthly_savings`
- `order_how` – `asc` or `desc` (default `desc` for utilization)
- `limit` (1–100, default 20), `offset` (default 0)

## Example request:

```
curl -s -H "x-rh-identity: ${IDENTITY}" \
  'http://localhost:8000/api/cost-
  management/v1/recommendations/openshift/quota/?filter[recommendation_type]=tighten&limit=10'
```

## Example response snippet:

```
{
  "meta": { "count": 1, "limit": 10, "offset": 0, "currency": "USD" },
  "links": { "first": "...", "last": "..." },
  "data": [
    {
      "cluster_uuid": "550e8400-e29b-41d4-a716-446655440001",
      "namespace": "production",
      "quota_name": "compute-quota",
      "recommendation_type": "tighten",
      "risk_level": "low",
      "quota_hard": {
        "cpu_request_millicores": 100000,
        "cpu_limit_millicores": null,
        "memory_request_bytes": null,
        "memory_limit_bytes": null
      },
      "quota_used": { "cpu_request_millicores": 25000 },
      "quota_recommended": { "cpu_request_millicores": 36000 },
      "utilization": { "cpu_request_percent": 25.0 },
      "capacity_freed": { "cpu_millicores": 64000, "memory_bytes": 0 },
      "estimated_savings": { "value": "12.340000", "units": "USD" },
      "last_observed_at": "2026-05-28T12:00:00Z"
    }
  ]
}
```

## Notification codes

Namespace ResourceQuota rows may emit notification codes **70–72**. Filter the catalog with `GET /api/cost-management/v1/recommendations/openshift/notification-codes/?filter[plugin]=quota`. Code **73** (`CLUSTER_QUOTA_AT_CAPACITY`) applies to ClusterResourceQuota rows only.

| Code | Name                | Description                                                    |
|------|---------------------|----------------------------------------------------------------|
| 70   | QUOTA_NEAR_CAPACITY | Quota at high utilization risk (near capacity)                 |
| 71   | QUOTA_OVERSIZED     | Tighten recommendation – quota has significant unused headroom |
| 72   | QUOTA_BLOCKING      | Quota at or over hard limit – pods may be pending              |

## Detail and history

`GET /api/cost-management/v1/recommendations/openshift/quota/detail` returns one recommendation object (not wrapped in `data`). Required query parameters: `cluster_uuid`, `namespace` (alias `filter[project]`). Optional: `quota_name` (alias `filter[resource_quota_name]`) when multiple ResourceQuota objects exist in the namespace.

The response includes the same fields as a list row plus `headroom_basis_points` and a `history[]` array of per-resource snapshots for trend charts. Each history entry includes `recorded_at`, `resource`, `recommendation_type`, `risk_level`, and optional `recommended_hard`, `current_hard`, `current_used`, and `utilization_percent` from the previous observation.

Example detail request:

```
curl -s -H "x-rh-identity: ${IDENTITY}" \
  'http://localhost:8000/api/cost-management/v1/recommendations/openshift/quota/detail?cluster_uuid=550e8400-e29b-41d4-a716-446655440001&namespace=production&quota_name=compute-quota'
```

Requires the `quota` plugin (`ROS_ENABLED_PLUGINS`); when disabled the route returns `404`.

Quota recommendation settings (per-organization thresholds; env vars lock fields in `locked_fields`):

```
GET /api/cost-management/v1/recommendations/openshift/settings/quota
PUT /api/cost-management/v1/recommendations/openshift/settings/quota
DELETE /api/cost-management/v1/recommendations/openshift/settings/quota
```

Example GET/PUT body:

```
{
  "headroom_percent": 10,
  "high_risk_threshold_percent": 90,
  "medium_risk_threshold_percent": 70
}
```

PUT validation: `headroom_percent` 0-100; `high_risk_threshold_percent` 1-100 and greater than `medium_risk_threshold_percent`; `medium_risk_threshold_percent` 1-99. DELETE removes per-org overrides (falls back to `ROS_QUOTA_*` env or defaults 10/90/70).

## ClusterResourceQuota recommendations

OpenShift-only team/tenant **ClusterResourceQuota** right-sizing (distinct from namespace `ResourceQuota` and namespace usage recommendations). Compares CRQ hard/used metrics from `openshift_clusterresourcequota_usage` against aggregated namespace quota recommendation totals.

```
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/?filter[cluster]=550e8400-e29b-41d4-a716-446655440000
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/?filter[cluster_quota_name]=team-payments-quota
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/?filter[project]=team-payments-ns
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/?filter[recommendation_type]=tighten&filter[risk_level]=high
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/?group_by[cluster]=*&limit=20&offset=0
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/?order_by=utilization&order_how=desc&limit=20&offset=0
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/detail?cluster_uuid=550e8400-e29b-41d4-a716-446655440001&cluster_quota_name=team-payments-quota
```

Available filters:

- `filter[cluster]` – cluster UUID (exact match)
- `filter[cluster_quota_name]` – CRQ object name (aliases: `filter[cluster_resource_quota]`, `filter[crq]`)
- `filter[project]` – namespace that belongs to the CRQ selector (`filter[namespace]` accepted as alias)
- `filter[recommendation_type]` – `tighten`, `raise`, `optimal`, or `none`
- `filter[risk_level]` – `high`, `medium`, `low`, or `none`
- `group_by[cluster]` – aggregate rows per cluster (returns `count`, summed `estimated_savings`, and summed `capacity_freed` per cluster; mutually exclusive with per-CRQ list fields such as `cluster_quota_name`)
- `order_by` – `cluster_quota_name`, `utilization`, `risk_level`, or `estimated_monthly_savings`
- `order_how` – `asc` or `desc` (default `desc` when `order_by` is set)
- `limit` (1-100, default 20), `offset` (default 0)

Detail endpoint returns one CRQ recommendation object (not wrapped in `data`). The response includes a `history` array of per-resource snapshots (`recorded_at`, `resource`, `recommendation_type`, `risk_level`, and optional `hard/used/utilization` fields) for trend charts.

Example request:

```
curl -s -H "x-rh-identity: ${IDENTITY}" \
```

```
'http://localhost:8000/api/cost-management/v1/recommendations/openshift/cluster-quota/?filter[recommendation_type]=tighten&limit=10'
```

Example response snippet:

```
{
  "meta": { "count": 1, "limit": 10, "offset": 0 },
  "links": { "first": "...", "last": "..." },
  "data": [
    {
      "cluster_uuid": "550e8400-e29b-41d4-a716-446655440001",
      "cluster_quota_name": "team-payments-quota",
      "namespaces": ["namespace-a", "namespace-b"],
      "recommendation_type": "tighten",
      "risk_level": "low",
      "quota_hard": {
        "cpu_request_millicores": 500000,
        "memory_request_bytes": 1099511627776,
        "storage_request_bytes": 107374182400,
        "pods": 500
      },
      "quota_used": {
        "cpu_request_millicores": 175000,
        "storage_request_bytes": 53687091200,
        "pods": 120
      },
      "quota_recommended": {
        "cpu_request_millicores": 396000,
        "memory_request_bytes": 496125722624,
        "storage_request_bytes": 85899345920,
        "pods": 400
      },
      "utilization": {
        "cpu_request_percent": 35,
        "memory_request_percent": 12,
        "storage_request_percent": 50,
        "pods_percent": 24
      },
      "capacity_freed": {
        "cpu_cores_freed": 104,
        "memory_bytes": 603387187152,
        "storage_request_bytes": 21474836480,
        "pods_freed": 100
      },
      "estimated_savings": { "value": 420, "units": "USD" }
    }
  ]
}
```

Requires the `cluster-quota` plugin (`ROS_ENABLED_PLUGINS`); when disabled the route returns `404`. Clusters without `ClusterResourceQuota` objects return an empty `data` array (not an error).

## Notification codes

CRQ rows may emit notification codes **70–73**. Filter the catalog with `GET /api/cost-management/v1/recommendations/openshift/notification-codes/?filter[plugin]=cluster-quota`.

| Code | Name                      | Description                          |
|------|---------------------------|--------------------------------------|
| 70   | QUOTA_NEAR_CAPACITY       | CRQ at high utilization risk         |
| 71   | QUOTA_OVERSIZED           | CRQ tighten recommendation generated |
| 72   | QUOTA_BLOCKING            | CRQ resource blocking (used >= hard) |
| 73   | CLUSTER_QUOTA_AT_CAPACITY | CRQ at capacity alert                |

Object-count resources (`count/deployments.apps`, `count/secrets`, etc.) contribute to `risk_level` and blocking notifications (code 72) only – they are not exposed in API `utilization` fields or used in `order_by=utilization`.

When Koku cost model rates change, masu triggers `POST /api/cost-management/v1/internal/recalculate-savings` with `recommendation_types` including `cluster-quota` to refresh `estimated_savings` on existing tighten rows.

Example `group_by[cluster]` response snippet:

```
{
  "meta": { "count": 2, "limit": 20, "offset": 0 },
  "data": [
    {
      "cluster_uuid": "550e8400-e29b-41d4-a716-446655440001",
      "count": 3,
      "capacity_freed": {
        "cpu_cores_freed": 12,
        "memory_bytes": 85899345920,
        "storage_request_bytes": 10737418240,
        "pods_freed": 5
      },
      "estimated_savings": { "value": 840, "units": "USD" }
    }
  ]
}
```

Cluster-quota recommendation settings (per-organization thresholds; env vars lock fields in `locked_fields`):

```
GET /api/cost-management/v1/recommendations/openshift/settings/cluster-quota
PUT /api/cost-management/v1/recommendations/openshift/settings/cluster-quota
DELETE /api/cost-management/v1/recommendations/openshift/settings/cluster-quota
```

Example GET/PUT body:

```
{
  "headroom_percent": 10,
```

```
"high_risk_threshold_percent": 90,
"medium_risk_threshold_percent": 70
}
```

PUT validation matches namespace quota settings. DELETE removes per-org overrides (falls back to `ROS_CLUSTER_QUOTA_*` env or defaults 10/90/70).

## Snapshot staleness recommendations

VolumeSnapshot analysis – orphaned, stale, never-restored, redundant snapshots with estimated storage savings:

```
GET /api/cost-management/v1/recommendations/openshift/snapshots
```

Namespace/cluster rollup for prioritizing cleanup (reclaimable cost and size exclude `active` and `managed`):

```
GET /api/cost-management/v1/recommendations/openshift/snapshots/summary
GET /api/cost-management/v1/recommendations/openshift/snapshots/summary?group_by=project
GET /api/cost-management/v1/recommendations/openshift/snapshots/summary?group_by=cluster
GET /api/cost-
management/v1/recommendations/openshift/snapshots/summary?filter[cluster]=CLUSTER_UUID
GET /api/cost-management/v1/recommendations/openshift/snapshots/summary?filter[project]=payments-
*
GET /api/cost-
management/v1/recommendations/openshift/snapshots/summary?order_by=reclaimable_restore_size_gib&o
rder_how=desc&limit=20
```

Snapshot staleness settings (configurable thresholds):

```
GET /api/cost-management/v1/recommendations/openshift/settings/snapshot
PUT /api/cost-management/v1/recommendations/openshift/settings/snapshot
DELETE /api/cost-management/v1/recommendations/openshift/settings/snapshot
```

## OpenShift Virtualization (VM) recommendations

VM-level CPU, memory, disk, and GPU sizing for KubeVirt workloads. Requires the ROS `vm` plugin (`ROS_VM_RECOMMENDATIONS_ENABLED` / Helm `enabledPlugins` includes `vm`). When disabled, VM routes return `404`.

Monthly `savings` on list/detail requires `ROS_SAVINGS_ESTIMATES_ENABLED=true` (default) and `KOKU_MASU_URL` with cost model rates at ingestion. When disabled or masu is unreachable, `savings` is JSON `null`. Values are fixed at generation time until the next recommendation cycle (rate or currency changes are not retroactive).

### VM recommendations list

```
GET /api/cost-management/v1/recommendations/openshift/vm
GET /api/cost-management/v1/recommendations/openshift/vm?limit=20&offset=0
GET /api/cost-management/v1/recommendations/openshift/vm?filter[cluster]=CLUSTER_UUID
GET /api/cost-management/v1/recommendations/openshift/vm?filter[project]=my-namespace
GET /api/cost-management/v1/recommendations/openshift/vm?filter[vm_name]=my-vm
```

```
GET /api/cost-management/v1/recommendations/openshift/vm?filter[term]=medium_term
GET /api/cost-management/v1/recommendations/openshift/vm?filter[confidence]=high
GET /api/cost-management/v1/recommendations/openshift/vm?filter[is_idle]=true
GET /api/cost-management/v1/recommendations/openshift/vm?filter[is_abandoned]=true
GET /api/cost-management/v1/recommendations/openshift/vm?filter[is_oversized]=true
GET /api/cost-management/v1/recommendations/openshift/vm?filter[guest_agent_detected]=true
GET /api/cost-management/v1/recommendations/openshift/vm?filter[has_gpu]=true
GET /api/cost-management/v1/recommendations/openshift/vm?filter[gpu_classification]=underutilized
GET /api/cost-management/v1/recommendations/openshift/vm?filter[engine]=cost
GET /api/cost-management/v1/recommendations/openshift/vm?filter[engine]=performance
```

Each list row includes `metadata.engine` matching the filter (one row per VM per engine when unfiltered).

Pagination: `limit` (1–100, default 10), `offset` (default 0).

Filters (`filter[...]` or legacy query names where noted):

| Filter                                                                                                                                  | Values / notes                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <code>cluster</code>                                                                                                                    | Cluster UUID (RBAC-scoped)                                                         |
| <code>namespace</code>                                                                                                                  | VM namespace                                                                       |
| <code>vm_name</code>                                                                                                                    | KubeVirt VM name                                                                   |
| <code>term</code>                                                                                                                       | <code>short_term</code> , <code>medium_term</code> , or <code>long_term</code>     |
| <code>engine</code>                                                                                                                     | <code>cost</code> or <code>performance</code>                                      |
| <code>confidence</code>                                                                                                                 | <code>high</code> , <code>moderate</code> , or <code>low</code>                    |
| <code>is_idle</code> , <code>is_abandoned</code> , <code>is_oversized</code> , <code>guest_agent_detected</code> , <code>has_gpu</code> | <code>true</code> or <code>false</code>                                            |
| <code>gpu_classification</code>                                                                                                         | Comma-separated GPU classes (e.g. <code>idle</code> , <code>underutilized</code> ) |

Each list item includes `metadata.is_network_bound` (`true` when sustained network metrics qualify the VM for **n1** network-optimized instance types). Use `filter[is_network_bound]=true` or `false` to narrow the list server-side.

Placement metadata (no list filters yet): `metadata.is_redundant_placement` (code **60**), `metadata.has_shared_storage` (code **62**), `metadata.numa_oversized` (code **63**). Code **61** is uneven node distribution (notification only).

`order_by` (default `vm_name`): `vm_name`, `namespace`, `current_vcpu`, `current_memory_gib`, `guest_os`, `recommended_vcpu`, `recommended_memory_gib`, `is_idle`, `is_abandoned`, `is_oversized`, `confidence`, `last_recommended_at`. Use `order_how=asc` or `desc`.

## VM detail

```
GET /api/cost-management/v1/recommendations/openshift/vm/detail?cluster_uuid=UUID&vm_name=NAME&namespace=NS
GET /api/cost-management/v1/recommendations/openshift/vm/detail?cluster_uuid=UUID&vm_name=NAME&namespace=NS&term=medium_term&engine=cost
```

Required: `cluster_uuid`, `vm_name`, `namespace`. Optional: `term` (default `medium_term`), `engine` (default `cost`).

`io_profile.pattern` classifies disk I/O from peak p95 throughput and IOPS: `sequential`, `random`, `mixed`, `low-io`, or empty when metrics are missing. Codes **58** and **59** align with `sequential` and `random` patterns.

Example detail response (CPU/memory/disk sizing in `current` / `recommended`; GPU in `gpu`; time series in `daily_digests`):

```
{
  "vm_name": "my-vm",
  "namespace": "my-namespace",
  "cluster_uuid": "550e8400-e29b-41d4-a716-446655440000",
  "guest_os": "linux",
  "current": {
    "vcpu": 4,
    "memory_gib": 16,
    "disk_gib": 100,
    "instance_type": "u1.medium"
  },
  "recommended": {
    "vcpu": 2,
    "memory_gib": 8,
    "disk_gib": 100,
    "instance_type": "n1.medium",
    "series": "n1"
  },
  "metadata": {
    "guest_agent_detected": true,
    "confidence": "high",
    "term": "medium_term",
    "engine": "cost",
    "is_idle": false,
    "is_abandoned": false,
    "is_oversized": true,
    "is_network_bound": true,
    "is_redundant_placement": false,
    "has_shared_storage": false,
    "numa_oversized": false,
    "preference_name": "small",
    "preference_class": "virtualmachineclusterpreference"
  },
  "io_profile": {
    "read_iops_p95": 1200,
    "write_iops_p95": 800,
    "read_bps_p95": 125829120,
    "write_bps_p95": 83886080,
    "hint": "balanced",
    "pattern": "sequential"
  },
  "disk_projection": {
    "days_until_full": 45,
    "growth_gib_per_day": 0.12,
    "recommended_expand_gib": 0
  },
  "notifications": [
    {
      "code": 55,
      "type": "warning",
      "message": "Network-saturated workload: recommend n1 network-optimized instance type"
    }
  ],
  "savings": {
    "value": "18104.000000",
  }
}
```

```

    "units": "USD"
  },
  "gpu": {
    "gpu_count": 1,
    "gpu_model": "NVIDIA-A100",
    "gpu_classification": "underutilized",
    "recommended_gpu_action": "time_slice",
    "recommended_time_slice_count": 4,
    "gpu_timeslice_confidence": "high",
    "gpu_timeslice_rationale": "7 days of observations; peak SM/DRAM/FB fractions support 4 replicas",
    "recommended_vgpu_profile": "nvidia.com/gpu.a100-4c.8g",
    "gpu_devices": []
  },
  "last_recommended_at": "2026-05-01T12:00:00Z",
  "daily_digests": [
    {
      "bucket_date": "2026-04-30",
      "cpu_usage_p95_mc": 1800,
      "mem_usage_p95_kib": 8388608,
      "sample_count": 24
    }
  ]
}

```

## VM recommendation history

Per-VM history of past sizing recommendations:

```

GET /api/cost-management/v1/recommendations/openshift/vms/{vm_name}/history?cluster_uuid=UUID&namespace=NS&limit=20&offset=0
GET /api/cost-management/v1/recommendations/openshift/vms/{vm_name}/history?cluster_uuid=UUID&namespace=NS&term=short_term&engine=cost

```

Required query params: `cluster_uuid` (or `cluster_id`), `namespace`. Path param: `vm_name`. Optional: `term` (default `short_term`), `engine` (default `cost`), `limit`, `offset`.

## VM instance types

Catalog of instance types and VM preference summary for a cluster:

```

GET /api/cost-management/v1/recommendations/openshift/instance-types?cluster_uuid=UUID

```

## VM settings

Dedicated VM threshold, disk, I/O, memory floor, and stability settings (not the generic `/settings/thresholds` endpoint):

```
GET /api/cost-management/v1/recommendations/openshift/settings/vm
PUT /api/cost-management/v1/recommendations/openshift/settings/vm
DELETE /api/cost-management/v1/recommendations/openshift/settings/vm
```

Example GET response:

```
{
  "enabled": true,
  "cpu_adaptive_margin_enabled": true,
  "history_retention_days": 90,
  "thresholds": {
    "cpu_percentile_cost": 0.60,
    "cpu_percentile_perf": 0.98,
    "cpu_margin_min": 1.15,
    "cpu_margin_max": 1.50,
    "mem_margin_min": 1.10,
    "downsize_hysteresis_ratio": 0.85,
    "min_vcpu_change": 1,
    "min_gib_change": 1,
    "idle_cpu_mc": 50,
    "idle_memory_mib": 256,
    "idle_cpu_mc_windows": 100,
    "idle_memory_mib_windows": 512,
    "abandoned_min_days": 14
  },
  "memory_floors": {
    "linux_gib": 2,
    "windows_gib": 4,
    "windows_kernel_reserve_gib": 0.5
  },
  "stability": {
    "downsize_stability_days": 3,
    "crash_loop_restart_threshold": 5
  },
  "disk": {
    "projection_window_days": 14,
    "headroom_pct": 0.20,
    "round_step_gib": 10,
    "min_growth_mib_per_day": 10
  },
  "io": {
    "high_iops_threshold": 3000,
    "sequential_threshold_bytes": 65536,
    "random_threshold_bytes": 16384,
    "min_iops_for_classification": 100
  },
  "network": {
    "throughput_threshold_bps": 62500000,
    "pps_threshold": 100000,
    "drop_ratio_bp": 10,
    "sustained_days": 7,
    "enable_network_series": true
  },
  "gpu": {
```

```

    "idle_threshold_bp": 500,
    "underutil_threshold_bp": 3000,
    "fb_saturation_mib": 0,
    "compute_saturation_threshold_bp": 8500,
    "gpu_timeslice_min_replicas": 2,
    "gpu_timeslice_max_replicas": 16,
    "gpu_timeslice_fb_safety_threshold_bp": 8000,
    "gpu_timeslice_dram_penalty_threshold_bp": 5000
  },
  "placement": {
    "enable_placement_checks": true,
    "placement_skew_ratio": 3,
    "enable_shared_pvc_correlation": true,
    "numa_node_memory_gib": 64
  },
  "instance_type_matching": true,
  "locked_fields": [],
  "settings_locked": false
}

```

**placement** – cluster placement, correlated workloads, and NUMA fit (notifications **60–63**):

- **enable\_placement\_checks** – same-node redundancy and node skew (default **true**)
- **placement\_skew\_ratio** – max:min VM count per node before skew notification **61** (default **3**)
- **enable\_shared\_pvc\_correlation** – correlated profile peers in namespace (default **true**)
- **numa\_node\_memory\_gib** – single-NUMA memory cap for code **63** (default **64**)

Env locks: **ROS\_VM\_ENABLE\_PLACEMENT\_CHECKS**, **ROS\_VM\_PLACEMENT\_SKEW\_RATIO**,  
**ROS\_VM\_ENABLE\_SHARED\_PVC\_CORRELATION**, **ROS\_VM\_NUMA\_NODE\_MEMORY\_GIB**.

**network** – n1 network-optimized classification (defaults shown above):

- **throughput\_threshold\_bps** – sustained aggregate rx+tx P95 above this (bytes/sec; default **62500000** ■ **500** Mbps)
- **pps\_threshold** – packets-per-second P95 threshold for alternate network-bound path (default **100000**)
- **drop\_ratio\_bp** – max daily drop ratio must exceed this in basis points (default **10** = 0.1%) with high PPS
- **sustained\_days** – days in the term window meeting throughput or PPS+drop criteria (default **7**)
- **enable\_network\_series** – when **true**, network-bound VMs may receive **n1.\*** instance types (default **true**)

**io** – disk I/O profiling and pattern classification:

- **high\_iops\_threshold** – total read+write p95 IOPS above this triggers code **39** (**VM\_HIGH\_IO**) (default **3000**)
- **sequential\_threshold\_bytes** – average I/O size (peak BPS / peak IOPS) ■ this ■ **io\_profile.pattern** **sequential** and code **58** (default **65536** = 64 KiB)
- **random\_threshold\_bytes** – average I/O size < this ■ **random** and code **59** (default **16384** = 16 KiB); values between random and sequential thresholds yield **mixed**
- **min\_iops\_for\_classification** – peak IOPS below this ■ **low-io** (no pattern codes) (default **100**)

`gpu` – VM GPU classification and vGPU time-slicing tunables:

- `idle_threshold_bp` – SM utilization below this ■ idle classification (default `500` = 5%)
- `underutil_threshold_bp` – below this but above idle ■ underutilized (default `3000` = 30%)
- `fb_saturation_mib` – frame-buffer saturation threshold in MiB; `0` auto-detects from GPU catalog (default `0`)
- `compute_saturation_threshold_bp` – SM fraction at or above this ■ compute-saturated (default `8500` = 85%)
- `gpu_timeslice_min_replicas` – minimum `recommended_time_slice_count` when time-slicing is advised (default `2`)
- `gpu_timeslice_max_replicas` – upper cap on slice count; reduced when DRAM exceeds penalty threshold (default `16`)
- `gpu_timeslice_fb_safety_threshold_bp` – do not recommend time-slicing when frame-buffer fraction ■ this (default `8000` = 80%)
- `gpu_timeslice_dram_penalty_threshold_bp` – when DRAM ■ this, max replicas is halved (default `5000` = 50%)

Detail responses may include GPU fields `gpu_timeslice_confidence` (`high`, `moderate`, `low`, or empty), `gpu_timeslice_rationale`, and `recommended_vgpu_profile` when time-slicing or vGPU profile selection applies.

Example PUT (partial update – I/O pattern, GPU classification, network, and time-slicing):

```
{
  "thresholds": {
    "cpu_margin_min": 1.18
  },
  "io": {
    "sequential_threshold_bytes": 131072,
    "random_threshold_bytes": 8192,
    "min_iops_for_classification": 200
  },
  "network": {
    "throughput_threshold_bps": 100000000,
    "sustained_days": 5
  },
  "gpu": {
    "idle_threshold_bp": 300,
    "underutil_threshold_bp": 2500,
    "compute_saturation_threshold_bp": 9000,
    "gpu_timeslice_max_replicas": 8,
    "gpu_timeslice_fb_safety_threshold_bp": 7500
  }
}
```

`history_retention_days` is read-only (set by `ROS_VM_REC_HISTORY_RETENTION_DAYS`). DELETE removes per-org VM settings overrides (`204 No Content`).

## VM notification codes (55–63)

VM list/detail `notifications` arrays use lowercase `type` (`info`, `warning`, `critical`) and numeric `code`:

```
| Code | Name | Severity | When emitted | |-----|-----|-----|-----| | 55 | VM_NETWORK_SATURATED |
warning | metadata.is_network_bound is true – recommend n1 network-optimized instance type | | 56 |
VM_VGPU_PROFILE_RECOMMENDED | info | vGPU profile recommended – see recommended_vgpu_profile in the gpu
object | | 57 | VM_GPU_TIMESLICE_UNSAFE_FB | warning | Frame-buffer usage too high for safe shared vGPU time-
slicing | | 58 | VM_IO_SEQUENTIAL | info | io_profile.pattern is sequential – consider storage optimized for
throughput | | 59 | VM_IO_RANDOM | info | io_profile.pattern is random – consider storage optimized for IOPS | | 60
| VM_REDUNDANT_COLOCATION | warning | metadata.is_redundant_placement is true – same-profile peers on same
node | | 61 | VM_UNEVEN_NODE_DISTRIBUTION | info | Uneven spread of profile VMs across nodes | | 62 |
VM_SHARED_STORAGE | info | metadata.has_shared_storage is true – correlated workload group | | 63 |
VM_NUMA_OVERSIZED | warning | metadata.numa_oversized is true – memory exceeds
placement.numa_node_memory_gib |
```

## VM term settings

Separate from generic `/settings/terms` – controls short/medium/long windows for VM recommendations only:

```
GET /api/cost-management/v1/recommendations/openshift/settings/vm/terms
PUT /api/cost-management/v1/recommendations/openshift/settings/vm/terms
DELETE /api/cost-management/v1/recommendations/openshift/settings/vm/terms
```

Example GET response:

```
{
  "terms": [
    {
      "name": "short_term",
      "window_days": 1,
      "min_data_days": 1,
      "locked": false,
      "is_default": true
    },
    {
      "name": "medium_term",
      "window_days": 7,
      "min_data_days": 3,
      "locked": false,
      "is_default": true
    },
    {
      "name": "long_term",
      "window_days": 15,
      "min_data_days": 7,
      "locked": false,
      "is_default": true
    }
  ],
  "settings_locked": false
}
```

## Recommendation term settings

Customers can configure their own 3 recommendation term windows (1–90 days each), replacing the fixed short (1d) / medium (7d) / long (15d) defaults:

```
GET /api/cost-management/v1/recommendations/openshift/settings/terms?recommendation_type={container|namespace|node|gpu|pvc}
PUT /api/cost-management/v1/recommendations/openshift/settings/terms?recommendation_type={container|namespace|node|gpu|pvc}
DELETE /api/cost-management/v1/recommendations/openshift/settings/terms?recommendation_type={container|namespace|node|gpu|pvc}
```

Required query parameter `recommendation_type` selects the plugin whose short/medium/long windows are read or updated. VM term windows use `/settings/vm/terms` instead. Use `GET /settings/capabilities` to see which types are enabled and support terms.

## Settings capabilities

Discover which recommendation plugins are enabled for the deployment and whether each supports configurable terms. Use each item's `name` as `recommendation_type` for `/settings/terms`. The top-level `business_hours` flag indicates whether business-hours settings endpoints are available (`ROS_BUSINESS_HOURS_ENABLED`).

```
GET /api/cost-management/v1/recommendations/openshift/settings/capabilities
```

GET response:

```
{
  "recommendation_types": [
    {
      "name": "container",
      "enabled": true,
      "supports_terms": true
    },
    {
      "name": "namespace",
      "enabled": true,
      "supports_terms": true
    },
    {
      "name": "pvc",
      "enabled": true,
      "supports_terms": true
    }
  ],
  "business_hours": false
}
```

## Platform settings lock

When `ROS_SETTINGS_LOCKED=true`, tenant settings overrides are ignored on read and all settings `PUT/DELETE` requests return `403 Forbidden`:

```
{
  "error": "settings are locked by platform administrator",
  "locked": true
}
```

GET responses include `settings_locked: true` and `locked_fields: ["*"]`. Per-plugin opt-outs use `ROS_SETTINGS_LOCKED_VM`, `ROS_SETTINGS_LOCKED_TERMS`, etc. VM sizing thresholds use the dedicated `/settings/vm` endpoint (not `/settings/thresholds?recommendation_type=vm`).

## Recommendation threshold settings

Per-organization sizing and classification thresholds for container, namespace, node, GPU, and PVC plugins. OpenShift Virtualization (VM) thresholds are configured via `/settings/vm` (see [OpenShift Virtualization \(VM\) recommendations](#)), not these endpoints.

Each plugin has a dedicated path (no query parameter). `PUT` accepts partial updates; `DELETE` removes tenant overrides so environment and compiled defaults apply. Fields locked by environment variables are listed in `locked_fields` and return `403` if modified via the API.

```
GET /api/cost-management/v1/recommendations/openshift/settings/container
PUT /api/cost-management/v1/recommendations/openshift/settings/container
DELETE /api/cost-management/v1/recommendations/openshift/settings/container

GET /api/cost-management/v1/recommendations/openshift/settings/namespace
PUT /api/cost-management/v1/recommendations/openshift/settings/namespace
DELETE /api/cost-management/v1/recommendations/openshift/settings/namespace

GET /api/cost-management/v1/recommendations/openshift/settings/node
PUT /api/cost-management/v1/recommendations/openshift/settings/node
DELETE /api/cost-management/v1/recommendations/openshift/settings/node

GET /api/cost-management/v1/recommendations/openshift/settings/gpu
PUT /api/cost-management/v1/recommendations/openshift/settings/gpu
DELETE /api/cost-management/v1/recommendations/openshift/settings/gpu

GET /api/cost-management/v1/recommendations/openshift/settings/pvc
PUT /api/cost-management/v1/recommendations/openshift/settings/pvc
DELETE /api/cost-management/v1/recommendations/openshift/settings/pvc
```

**Deprecated alias** (still supported; responses include `Deprecation: true` and a `Link` header pointing to the canonical path above):

```
GET /api/cost-
management/v1/recommendations/openshift/settings/thresholds?recommendation_type={container|namesp
```

```
ace|node|gpu|pvc}
PUT /api/cost-
management/v1/recommendations/openshift/settings/thresholds?recommendation_type={container|namesp
ace|node|gpu|pvc}
DELETE /api/cost-
management/v1/recommendations/openshift/settings/thresholds?recommendation_type={container|namesp
ace|node|gpu|pvc}
```

## Container and namespace (/settings/container or /settings/namespace)

Container and namespace share the same response shape. Namespace defaults use a higher

`mem_trend_slope_threshold` (500.0 vs 100.0 for containers).

GET/PUT response (/settings/container):

```
{
  "cpu_cost_percentile": 0.60,
  "cpu_perf_percentile": 0.98,
  "mem_cost_percentile": 0.95,
  "mem_perf_percentile": 1.0,
  "min_margin": 1.15,
  "max_margin": 1.50,
  "limit_multiplier": 1.05,
  "cpu_floor_mc": 25,
  "idle_cpu_threshold_mc": 10,
  "idle_mem_threshold_kib": 10240,
  "mem_trend_slope_threshold": 100.0,
  "low_confidence_threshold": 0.5,
  "locked_fields": []
}
```

GET response (/settings/namespace) – same fields, higher memory trend threshold:

```
{
  "cpu_cost_percentile": 0.60,
  "cpu_perf_percentile": 0.98,
  "mem_cost_percentile": 0.95,
  "mem_perf_percentile": 1.0,
  "min_margin": 1.15,
  "max_margin": 1.50,
  "limit_multiplier": 1.05,
  "cpu_floor_mc": 25,
  "idle_cpu_threshold_mc": 10,
  "idle_mem_threshold_kib": 10240,
  "mem_trend_slope_threshold": 500.0,
  "low_confidence_threshold": 0.5,
  "locked_fields": []
}
```

Example PUT (container – override CPU percentile):

```
{
  "cpu_cost_percentile": 0.65
}
```

## Node (/settings/node)

```
{
  "underutil_threshold": 0.30,
  "overcommit_threshold": 1.50,
  "allocatable_factor": 0.93,
  "stranded_imbalance_threshold": 0.60,
  "ema_alpha": 0.30,
  "cost_target_utilization": 0.80,
  "perf_target_utilization": 0.55,
  "perf_consolidation_headroom_multiplier": 2.0,
  "trend_min_days": 3,
  "zombie_cpu_p95_mc": 200,
  "zombie_max_pods": 5,
  "idle_cpu_util_pct": 10,
  "idle_mem_util_pct": 10,
  "idle_max_pods": 10,
  "pod_headroom_consolidation_gate": 0.15,
  "pod_headroom_notification_threshold": 0.10,
  "locked_fields": []
}
```

Example PUT (node – override underutilization threshold):

```
{
  "underutil_threshold": 0.35
}
```

## GPU (/settings/gpu)

```
{
  "idle_threshold": 0.02,
  "underutilized_sm_threshold": 0.25,
  "underutilized_tensor_threshold": 0.15,
  "membound_dram_threshold": 0.60,
  "membound_tensor_threshold": 0.15,
  "fb_headroom_factor": 1.20,
  "compute_bound_dram_threshold": 0.30,
  "mig_fb_percentile": 0.98,
  "confidence_days_tier1": 3,
  "confidence_days_tier2": 7,
  "confidence_days_tier3": 14,
  "spike_ratio_threshold": 5.0,
  "spike_confidence_penalty": 0.70,
  "no_profiling_confidence_factor": 0.50,
  "timeslicing_majority_threshold": 0.50,
  "timeslicing_min_replicas": 2,
}
```

```

    "timeslicing_max_replicas": 8,
    "timeslicing_base_penalty": 0.70,
    "timeslicing_impacted_weight": 0.30,
    "node_freshness_days": 7,
    "locked_fields": []
  }

```

Example PUT (GPU – override idle threshold):

```

{
  "idle_threshold": 0.03
}

```

## PVC (/settings/pvc)

```

{
  "oversized_threshold": 0.20,
  "near_full_threshold": 0.85,
  "min_trend_days": 7,
  "recommended_size_multiplier": 2,
  "min_recommended_gib": 1,
  "days_to_full_alert": 30,
  "locked_fields": []
}

```

Example PUT (PVC – override oversized threshold):

```

{
  "oversized_threshold": 0.25
}

```

## Business hours settings

Optional schedule-aware sizing (CPU requests/limits and memory requests/limits) (requires

`ROS_BUSINESS_HOURS_ENABLED` and `business_hours: true` in

`/recommendations/openshift/settings/capabilities`). Enabling a schedule triggers digest re-shipping for affected scopes and approximately doubles digest storage.

Org-default schedule:

```

GET /api/cost-management/v1/recommendations/openshift/settings/business-hours
PUT /api/cost-management/v1/recommendations/openshift/settings/business-hours
DELETE /api/cost-management/v1/recommendations/openshift/settings/business-hours

```

Cluster override (`:cluster_id` is the cluster UUID):

```

GET /api/cost-management/v1/recommendations/openshift/settings/business-
hours/clusters/:cluster_id

```

```
PUT /api/cost-management/v1/recommendations/openshift/settings/business-
hours/clusters/:cluster_id
DELETE /api/cost-management/v1/recommendations/openshift/settings/business-
hours/clusters/:cluster_id
```

Namespace override (most specific wins):

```
GET /api/cost-management/v1/recommendations/openshift/settings/business-
hours/clusters/:cluster_id/namespaces/:namespace
PUT /api/cost-management/v1/recommendations/openshift/settings/business-
hours/clusters/:cluster_id/namespaces/:namespace
DELETE /api/cost-management/v1/recommendations/openshift/settings/business-
hours/clusters/:cluster_id/namespaces/:namespace
```

Example PUT body:

```
{
  "timezone": "America/New_York",
  "schedule": {
    "days": ["monday", "tuesday", "wednesday", "thursday", "friday"],
    "start_time": "08:00",
    "end_time": "17:00"
  },
  "off_hours_weight": 0.0,
  "enabled": true
}
```

`days` uses lowercase English day names. `start_time` / `end_time` are 24-hour `HH:MM` (UTC interpretation per stored timezone). PUT responses may include a storage warning and `reship_status` while digests are rebuilt. Container and namespace recommendation payloads include a nested `business_hours` object when a schedule applies.

## Idle and zombie detection

Classifies containers as `active`, `idle` (low request-relative utilization), or `zombie` (near-zero CPU with sustained observation). Idle workloads expose waste and terminate guidance on list/detail responses; rightsizing savings are omitted when `idle_state` is not `active`.

### Idle detection settings

Per-organization idle detection configuration. PUT accepts partial updates inside `idle_detection`; DELETE removes tenant overrides and restores environment/compiled defaults. Fields locked by environment variables appear in `locked_fields` and return `403` if modified via the API.

```
GET /api/cost-management/v1/recommendations/openshift/settings/idle-detection
PUT /api/cost-management/v1/recommendations/openshift/settings/idle-detection
DELETE /api/cost-management/v1/recommendations/openshift/settings/idle-detection
```

GET response:

```
{
  "idle_detection": {
    "enabled": true,
    "thresholds": {
      "cpu_utilization_percent": 2,
      "memory_utilization_percent": 5,
      "burst_ratio": 10,
      "minimum_observation_days": 14,
      "gpu_sm_active_basis_points": 500,
      "gpu_dram_active_basis_points": 500
    },
    "exclusions": {
      "namespaces": ["kube-system", "openshift-*"],
      "workload_types": ["DaemonSet"]
    }
  },
  "locked_fields": []
}
```

Example PUT (enable idle detection with custom thresholds and exclusions):

```
{
  "idle_detection": {
    "enabled": true,
    "thresholds": {
      "cpu_utilization_percent": 2,
      "memory_utilization_percent": 5,
      "burst_ratio": 10,
      "minimum_observation_days": 14,
      "gpu_sm_active_basis_points": 500,
      "gpu_dram_active_basis_points": 500,
      "zombie_cpu_millicores": 1,
      "zombie_peak_millicores": 10
    },
    "exclusions": {
      "namespaces": ["kube-system", "openshift-*"],
      "workload_types": ["DaemonSet"]
    }
  },
  "locked_fields": []
}
```

## Container list response fields

When idle detection is enabled, each container recommendation row may include:

| Field              | Definition                                                                            |
|--------------------|---------------------------------------------------------------------------------------|
| idle_state         | Classification: <code>active</code> , <code>idle</code> , or <code>zombie</code>      |
| idle_since         | ISO date ( <code>YYYY-MM-DD</code> ) when the workload entered the current idle state |
| idle_duration_days | Days in the current <code>idle_state</code>                                           |

| Field                   | Definition                                                                                                                         |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| peak_cpu_millicores     | Peak CPU usage (millicores) observed in the observation window                                                                     |
| peak_memory_bytes       | Peak memory usage (bytes) observed in the observation window                                                                       |
| estimated_monthly_waste | Monthly cost of retained resources for idle/zombie workloads (same {value, units} shape as savings fields; 6-decimal string + USD) |
| idle_recommendation     | Terminate guidance when idle_state is idle or zombie (action, confidence, reason)                                                  |

Example list row (idle/zombie):

```
{
  "idle_state": "zombie",
  "idle_since": "2026-04-15",
  "idle_duration_days": 42,
  "peak_cpu_millicores": 2,
  "peak_memory_bytes": 52428800,
  "estimated_monthly_waste": {
    "value": "89.500000",
    "units": "USD"
  },
  "idle_recommendation": {
    "action": "terminate",
    "confidence": "high",
    "reason": "No meaningful CPU or memory activity for 42 days"
  }
}
```

Omit `idle_recommendation` when `idle_state=active`. Omit `estimated_monthly_waste` when savings estimates are disabled or cost data is unavailable. Omit `estimated_monthly_savings` when `idle_state` is not `active` (rightsizing delta is not actionable for idle workloads).

## GPU idle filter

Filter by GPU idle state (`active`, `idle`, or `zombie`; comma-separated values are ORed). GPU idle fields on container rows: `gpu_idle_state`, `gpu_idle_since`, `gpu_idle_duration_days`, `gpu_estimated_waste_cents`.

```
GET /api/cost-management/v1/recommendations/openshift?filter[gpu_idle_state]=idle,zombie
```

## Namespace idle filter

Filter namespaces by aggregated idle state (`active`, `idle`, or `zombie`).

```
GET /api/cost-management/v1/recommendations/openshift/namespaces?filter[idle_state]=idle,zombie
```

## Container list ordering (idle)

Sort idle workloads with `order_by` and `order_how` (`asc` or `desc`):

```
GET /api/cost-management/v1/recommendations/openshift?order_by=idle_state
GET /api/cost-management/v1/recommendations/openshift?order_by=idle_duration_days
GET /api/cost-management/v1/recommendations/openshift?order_by=estimated_monthly_waste
```

## PVC orphan idle

Orphaned PVC recommendations include `idle_since` and `idle_duration_days` fields tracking when zero usage was first detected.

## Idle notification codes

| Code | Name               | When                                                                              |
|------|--------------------|-----------------------------------------------------------------------------------|
| 5    | IDLE_WORKLOAD      | Container idle or zombie ( <code>idle_state</code> )                              |
| 8    | ABANDONED_WORKLOAD | Legacy abandoned workload (all-zero usage when inline classification did not run) |
| 15   | NODE_IDLE          | Node idle or zombie                                                               |
| 26   | GPU_IDLE           | GPU idle or zombie                                                                |

## Async recalculation

PUT and DELETE on idle-detection settings enqueue async threshold recalculation for plugins: **container**, **gpu**, **namespace**, **node**, and **pvc**.

## Savings summary by idle state

Group fleet idle waste by state (query value is ignored; use `*`):

```
GET /api/cost-management/v1/recommendations/openshift/savings-summary?group_by[idle_state]=*
```

Response (`FleetSavingsByIdleStateResponse`):

```
{
  "meta": { "count": 2 },
  "data": [
    {
      "idle_state": "zombie",
      "estimated_monthly_waste": { "value": "4200.000000", "units": "USD" },
      "container_count": 12
    },
    {
      "idle_state": "idle",
      "estimated_monthly_waste": { "value": "8250.000000", "units": "USD" },
    }
  ]
}
```

```
"container_count": 35
    }
  ]
}
```

Report rightsizing savings and idle waste separately on dashboards so totals are not double-counted (idle = eliminate workload; rightsizing = reduce requests).

## Recommendation history and quality

Historical tracking of recommendation changes over time:

```
GET /api/cost-management/v1/recommendations/openshift/history
```

Recommendation quality metrics (`stability_pct` 0.0–1.0, `adoption_detected`, `oom_events_after_rec`, and `recommendation_age_hours` – measuring how stable and adopted recommendations are over time):

```
GET /api/cost-management/v1/recommendations/openshift/quality
```

## Forecasts

Cost forecasts predict future spending based on historical trends.

### OpenShift cost forecast

```
GET /api/cost-management/v1/forecasts/openshift/costs/
```

### AWS cost forecast

```
GET /api/cost-management/v1/forecasts/aws/costs/
```

### Azure cost forecast

```
GET /api/cost-management/v1/forecasts/azure/costs/
```

### Google Cloud cost forecast

```
GET /api/cost-management/v1/forecasts/gcp/costs/
```

### OCP-on-cloud cost forecasts

```
GET /api/cost-management/v1/forecasts/openshift/infrastructures/all/costs/
```

```
GET /api/cost-management/v1/forecasts/openshift/infrastructures/aws/costs/
GET /api/cost-management/v1/forecasts/openshift/infrastructures/azure/costs/
GET /api/cost-management/v1/forecasts/openshift/infrastructures/gcp/costs/
```

## Resource Types

Resource type endpoints return the available values for group\_by and filter parameters. Useful for building dropdowns in UIs or discovering what filter values exist.

### OpenShift

```
GET /api/cost-management/v1/resource-types/openshift-clusters/
GET /api/cost-management/v1/resource-types/openshift-projects/
GET /api/cost-management/v1/resource-types/openshift-nodes/
GET /api/cost-management/v1/resource-types/openshift-virtual-machines/
GET /api/cost-management/v1/resource-types/openshift-gpu-vendors/
GET /api/cost-management/v1/resource-types/openshift-gpu-models/
```

### AWS

```
GET /api/cost-management/v1/resource-types/aws-accounts/
GET /api/cost-management/v1/resource-types/aws-services/
GET /api/cost-management/v1/resource-types/aws-regions/
GET /api/cost-management/v1/resource-types/aws-organizational-units/
GET /api/cost-management/v1/resource-types/aws-categories/
```

### Azure

```
GET /api/cost-management/v1/resource-types/azure-subscription-guids/
GET /api/cost-management/v1/resource-types/azure-services/
GET /api/cost-management/v1/resource-types/azure-regions/
```

### Google Cloud

```
GET /api/cost-management/v1/resource-types/gcp-accounts/
GET /api/cost-management/v1/resource-types/gcp-projects/
GET /api/cost-management/v1/resource-types/gcp-services/
GET /api/cost-management/v1/resource-types/gcp-regions/
```

### Cost models

```
GET /api/cost-management/v1/resource-types/cost-models/
```

## Cost Distribution Fields

OpenShift cost report responses include distributed overhead costs when a cost model has platform/worker distribution enabled:

| Field                               | Definition                                                                                         |
|-------------------------------------|----------------------------------------------------------------------------------------------------|
| cost_platform_distributed           | Platform project overhead distributed proportionally to user projects (by CPU or memory weighting) |
| cost_worker_unallocated_distributed | Worker node unallocated capacity distributed proportionally to user projects                       |
| cost_total                          | Total cost including raw + usage + markup + platform_distributed + worker_unallocated_distributed  |

These fields appear in nested form in the API JSON response (e.g. `cost.platform_distributed`, `cost.worker_unallocated_distributed`, `cost.total`).

## Settings

### Platform projects (cost groups)

```
GET /api/cost-management/v1/settings/cost-groups/
```

### Enabled tags

All enabled tags:

```
GET /api/cost-management/v1/settings/tags/?filter[enabled]=true
```

Enabled tags filtered by source type:

```
GET /api/cost-management/v1/settings/tags/?filter[enabled]=true&filter[source_type]=OCP
GET /api/cost-management/v1/settings/tags/?filter[enabled]=true&filter[source_type]=AWS
GET /api/cost-management/v1/settings/tags/?filter[enabled]=true&filter[source_type]=Azure
GET /api/cost-management/v1/settings/tags/?filter[enabled]=true&filter[source_type]=GCP
```

### Display currency

```
GET /api/cost-management/v1/account-settings/currency/
```

## Cost models

List all cost models:

```
GET /api/cost-management/v1/cost-models/
```

Get rates on a single cost model by adding the cost model UUID:

```
GET /api/cost-management/v1/cost-models/{cost_model_uuid}/
```

Cost model relations are set at an OpenShift integration level (which includes one OpenShift cluster), or AWS/Azure/GCP integration level (which includes the parent and all child accounts in the same cloud integration). Everything covered by one integration shares the same cost model.

An integration can only be assigned to a single cost model, but multiple integrations (e. g. OpenShift clusters) can be assigned to the same cost model.

Matching data back to a cost model requires looking at the `source_uuid` value for each line item in the report endpoints, then searching for it in the cost-models endpoint.

## Integrations

Formerly known as "sources", an integration is a cloud account or OpenShift cluster added to Cost Management. You can use the API to list sources added to your account or see last ingest completion times per source.

List all integrations:

```
GET /api/cost-management/v1/sources/
```

Get a specific integration:

```
GET /api/cost-management/v1/sources/{source_uuid}/
```

Check the stats on a particular integration:

```
GET /api/cost-management/v1/sources/{source_uuid}/stats/
```

## Filtering and Pagination

### Time scope filters

All cloud and OpenShift cost endpoints support the following filters:

**filter[resolution]**

`daily` or `monthly`. Sets report to daily or monthly breakdown.

**filter[time\_scope\_value] and filter[time\_scope\_units]**

Used together to set the scope value and unit for the returned data.

- `-10` or `-30` with `filter[time_scope_units]=day` for the last 10 or 30 days.
- `-1`, `-2`, or `-3` with `filter[time_scope_units]=month` for the last 1, 2, or 3 months.

You can also use explicit start and end dates:

```
?start_date=2022-09-01&end_date=2022-09-10
```

## Exclude filter

Exclude specific values from results. Works with the same dimensions as `filter` and `group_by`:

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[project]=*&exclude[project]=openshift-monitoring
GET /api/cost-management/v1/reports/aws/costs/?group_by[account]=*&exclude[account]=123456789012
```

## Pagination

All list endpoints support `limit` and `offset` for pagination:

```
?limit=10&offset=0
```

The response `meta` object includes `count` (total items) and `links` includes `first`, `next`, `previous`, `last` URLs.

## Get CSV or JSON Reports

Any of the report endpoints can return CSV format by setting the `Accept` header to `text/csv`.

Any of the report endpoints can return JSON format by setting the `Accept` header to `application/json`.

Current month aggregation by cluster:

```
curl --location -g \
  'https://console.redhat.com/api/cost-management/v1/reports/openshift/costs/?filter[time_scope_units]=month&filter[time_scope_value]=-1&filter[resolution]=monthly&group_by[cluster]=' \
  -H 'accept: text/csv' \
  -H 'Authorization: Bearer ACCESS_TOKEN' \
  -o output.csv
```

Current month aggregation by project:

```
curl --location -g \  
  'https://console.redhat.com/api/cost-  
management/v1/reports/openshift/costs/?filter[time_scope_units]=month&filter[time_scope_value]=-  
1&filter[resolution]=monthly&group_by[project]='*' \  
  -H 'accept: text/csv' \  
  -H 'Authorization: Bearer ACCESS_TOKEN' \  
  -o output.csv
```

Current month grouped per tag:

```
curl --location -g \  
  'https://console.redhat.com/api/cost-  
management/v1/reports/openshift/costs/?filter[time_scope_units]=month&filter[time_scope_value]=-  
1&filter[resolution]=monthly&group_by[tag:tag_key]=tag_value' \  
  -H 'accept: text/csv' \  
  -H 'Authorization: Bearer ACCESS_TOKEN' \  
  -o output.csv
```

Current month combined group\_by with filter (projects grouped, filtered by cluster and node):

```
curl --location -g \  
  'https://console.redhat.com/api/cost-  
management/v1/reports/openshift/costs/?filter[time_scope_units]=month&filter[time_scope_value]=-  
1&filter[resolution]=monthly&group_by[project]='*&filter[cluster]=my_cluster&filter[node]=my_node'  
  \  
  -H 'accept: text/csv' \  
  -H 'Authorization: Bearer ACCESS_TOKEN' \  
  -o output.csv
```

All of the above examples and additional parameter/filter options can be used with CSV export.

## Basic Field Definitions

| Field        | Definition                                                                                                                       |
|--------------|----------------------------------------------------------------------------------------------------------------------------------|
| sup_usage    | Cost calculations from rates defined in the price list within a cost model when selecting Supplementary as the Calculation type  |
| sup_raw      | Not currently being used in the cost management backend                                                                          |
| sup_markup   | Markup set in the cost model, applied against sup_usage                                                                          |
| sup_total    | Total combined sup_cost: sup_usage + sup_markup                                                                                  |
| infra_usage  | Cost calculations from rates defined in the price list within a cost model when selecting Infrastructure as the Calculation type |
| infra_raw    | Cost coming directly from a cloud bill                                                                                           |
| infra_markup | Markup set in the cost model, applied against infra_raw and infra_usage                                                          |

| Field                  | Definition                                                                                           |
|------------------------|------------------------------------------------------------------------------------------------------|
| infra_total            | Total combined infra_cost: infra_usage + infra_markup + infra_raw                                    |
| cost_usage             | Total cost of usage: sup_usage + infra_usage                                                         |
| cost_raw               | Total cost of raw: sup_raw + infra_raw                                                               |
| cost_markup            | Total cost of markup: sup_markup + infra_markup                                                      |
| cost_total             | Total combined cost: cost_usage + cost_markup + cost_raw                                             |
| cost_units             | Cost currency unit for all cost metrics                                                              |
| capacity               | Total hourly capacity of metric (Memory/core hours)                                                  |
| capacity_count         | Max cores/Mem for a cluster                                                                          |
| capacity_count_units   | Unit for count (e.g. Core)                                                                           |
| usage                  | Total hourly usage for a project/node/cluster and/or day/month depending on the query                |
| usage_units            | Unit for the usage/request/limit metrics (e.g. Core-hours)                                           |
| request                | Total hourly request for a project/node/cluster and/or day/month depending on the query              |
| request_unused         | Total hourly request that is not being utilised (difference between request and usage)               |
| request_unused_percent | Percent value for unused cluster request based on $(\text{unused\_request} / \text{capacity}) * 100$ |
| limit                  | Total hourly limit for a project/node/cluster and/or day/month depending on the query                |
| gpu_request            | Total GPU device requests for a project/node/cluster                                                 |
| gpu_limit              | Total GPU device limits for a project/node/cluster                                                   |
| gpu_usage              | Total GPU utilization for a project/node/cluster (in device-hours)                                   |